

Day 6: AI基础理论与国内大模型应用 >>

*今日时间安排

时段	时间	内容	形式
上午	9:00-10:30	大语言模型基础原理	理论学习
	10:45-12:00	Prompt Engineering技巧	实践练习
下午	14:00-15:30	DeepSeek API深度实践	动手编码
	15:45-17:00	豆包(火山引擎)API集成	项目实战
晚上	19:00-20:30	AI能力整合到现有项目	完整应用
	20:45-21:00	成果展示与总结	自测

√ √ 学习目标

今日完成后，你将能够：

- √ 理解LLM原理 - Transformer架构、注意力机制、Tokenization
- √ 掌握Prompt设计 - Zero-shot/Few-shot/CoT等提示策略
- √ 调用DeepSeek API - 对话、文本生成、代码解释等场景
- √ 接入豆包API - 多模态能力、角色扮演、知识问答
- √ 构建AI工具模块 - 封装统一的LLM调用接口
- √ 实现智能功能 - 文本摘要、翻译、问答、内容生成

** 详细学习内容

1. 大语言模型原理（1.5小时）

1.1 从神经网络到Transformer

发展历程：

2017年之前:

RNN/LSTM * 处理序列数据, 但存在长距离依赖问题

2017年6月:

+P Attention Is All You Need (Google)

- * 提出Transformer架构
- * 自注意力机制解决长距离依赖
- * 并行计算能力大幅提升

2018-2019:

GPT-1, GPT-2 (OpenAI)

BERT (Google)

- * 预训练+微调范式确立

2020-2022:

GPT-3 (1750亿参数)

- * 展现出强大的Few-shot学习能力
- * In-context Learning成为可能

2022年底至今:

ChatGPT引爆AI革命

- * RLHF (人类反馈强化学习)
- * 指令遵循能力大幅增强
- * 国内大模型百花齐放 (DeepSeek、文心一言、通义千问、豆包...)

□1.2 Transformer核心概念

自注意力机制 (Self-Attention):

```

# 简化的注意力机制演示
import numpy as np

def softmax(x):
    """Softmax函数"""
    e_x = np.exp(x - np.max(x)) # 数值稳定
    return e_x / e_x.sum(axis=-1, keepdims=True)

def attention(query, key, value):
    """
    缩放点积注意力

    Attention(Q, K, V) = softmax(QK^T / *d_k) V
    """
    d_k = query.shape[-1]

    # 计算注意力分数
    scores = np.matmul(query, key.T) / np.sqrt(d_k)

    # Softmax归一化
    attention_weights = softmax(scores)

    # 加权求和
    output = np.matmul(attention_weights, value)

    return output, attention_weights

# 示例: 句子 "I love AI"
words = ["I", "love", "AI"]

# 假设的词嵌入向量（实际是768维或更高）
embeddings = {
    "I": [1.0, 0.0, 0.0],
    "love": [0.0, 1.0, 0.0],
    "AI": [0.0, 0.0, 1.0]
}

*

Q = np.array([embeddings[w] for w in words]) # Query矩阵
K = np.array([embeddings[w] for w in words]) # Key矩阵
V = np.array([embeddings[w] for w in words]) # Value矩阵

output, weights = attention(Q, K, V)

print("注意力权重矩阵:")
print(weights)

# 输出示例:
# [[0.33 0.33 0.33] * "I" 关注所有词
#  [0.40 0.35 0.25] * "love" 更关注"I"和"love"

```

```
# [0.25 0.35 0.40]] * "AI" 更关注"love"和"AI"
```

多头注意力 (Multi-Head Attention):

```

class MultiHeadAttention:
    """多头注意力机制"""

    def __init__(self, d_model=512, num_heads=8):
        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads # 每个头的维度

        # 线性投影层（简化表示）
        self.W_q = np.random.randn(d_model, d_model) * 0.01
        self.W_k = np.random.randn(d_model, d_model) * 0.01
        self.W_v = np.random.randn(d_model, d_model) * 0.01
        self.W_o = np.random.randn(d_model, d_model) * 0.01

    def forward(self, x):
        batch_size, seq_len, _ = x.shape

        # 1. 线性投影到多个头
        Q = x @ self.W_q
        K = x @ self.W_k
        V = x @ self.W_v

        # 2. 分割成多个头
        # (batch, seq_len, d_model) -> (batch, num_heads, seq_len, d_k)
        Q = Q.reshape(batch_size, seq_len, self.num_heads, self.d_k).transpose(0, 2, 1, 3)
        K = K.reshape(batch_size, seq_len, self.num_heads, self.d_k).transpose(0, 2, 1, 3)
        V = V.reshape(batch_size, seq_len, self.num_heads, self.d_k).transpose(0, 2, 1, 3)

        # 3. 并行计算每个头的注意力
        # (使用缩放点积注意力)
        scores = (Q @ K.transpose(-2, -1)) / np.sqrt(self.d_k)
        attn_weights = softmax(scores)

        # 4. 加权求和
        context = attn_weights @ V

        # 5. 合并多头
        context = context.transpose(0, 2, 1, 3).reshape(batch_size, seq_len, self.d_model)

        # 6. 最终线性投影
        output = context @ self.W_o

        return output, attn_weights

```

为什么需要多头?

不同头可以关注不同类型的关系:

Head 1: 语法关系 (主谓宾)

Head 2: 语义相似度

Head 3: 指代消解 (代词指代什么)

...

□1.3 Tokenization 与输入处理

什么是Token?

文本 * Token化 * 数字ID * Embedding * 模型输入

示例: "我喜欢编程"

+-- 字级Token: ["我", "喜", "欢", "编", "程"] (5个token)

+-- 词级Token: ["我", "喜欢", "编程"] (3个token)

+-- BPE/Subword: ["我", "喜欢", "编", "程"] (可能不同)

常见Tokenizer:

- GPT系列: Byte Pair Encoding (BPE)
- LLaMA: SentencePiece (BPE变体)
- 中文模型: 通常结合字和词

Token限制与成本:

不同模型的上下文长度限制

```
MODEL_CONTEXT_LENGTHS = {  
    'gpt-4': 128000,      # 约10万字  
    'gpt-3.5-turbo': 16385,  
    'deepseek-chat': 65536, # 约5万字  
    'doubao-pro': 128000,  
    *
```

Token估算规则（粗略）

```
def estimate_tokens(text: str) -> int:  
    """  
    英文: *4字符 * 1 token  
    中文: *1.5-2字符 * 1 token  
    """  
    chinese_chars = sum(1 for c in text if '\u4e00' <= c <= '\u9fff')  
    other_chars = len(text) - chinese_chars  
  
    return int(chinese_chars / 1.5 + other_chars / 4)
```

使用tiktoken精确计算

```
import tiktoken  
  
def count_tokens_exact(text: str, model: str = "gpt-3.5-turbo") -> int:  
    encoding = tiktoken.encoding_for_model(model)  
    return len(encoding.encode(text))
```

成本计算示例

```
PRICING = {  
    'deepseek-chat': {  
        'input': 0.001, # 元/千token  
        'output': 0.002  
    },  
    'doubao-pro': {  
        'input': 0.004,  
        'output': 0.012  
    },  
    *  
    *
```

```
def calculate_cost(model: str, input_tokens: int, output_tokens: int) -> float:  
    pricing = PRICING[model]  
    input_cost = (input_tokens / 1000) * pricing['input']  
    output_cost = (output_tokens / 1000) * pricing['output']  
    return input_cost + output_cost
```

示例

```
text = "请帮我写一篇关于人工智能发展的文章，大约2000字..."  
tokens = count_tokens_exact(text)
```

```
print(f"输入约 {tokens* tokens}")
```

假设输出5000 tokens


```
cost = calculate_cost('deepseek-chat', tokens, 5000)
```

```
print(f"预估成本: *{cost:.4f}*")
```

□ 2. Prompt Engineering 技巧 (1.5小时)

□ 2.1 核心Prompt模式

Zero-shot Learning (零样本学习) :

直接给出任务描述, 无需示例

zero_shot_prompts = {

 "情感分析": ""

 请判断以下评论的情感倾向 (正面/负面/中性) :

 评论: 这家餐厅的服务态度很好, 菜品也很美味!

 情感:

 "",

 "文本摘要": ""

 请用一句话概括以下文章的主要内容:

 文章: 人工智能 (Artificial Intelligence, 简称AI) 是计算机科学的一个分支, 旨在开发能够模拟人类智能的系统和技术。近年来, 随着深度学习技术的突破, AI在图像识别、自然语言处理、自动驾驶等领域取得了显著进展。

 摘要:

 "",

 "代码生成": ""

 用Python编写一个快速排序算法的实现:

 要求:

1. 包含详细注释
2. 处理边界情况
3. 时间复杂度为 $O(n \log n)$

 代码:

 ""

 *

Few-shot Learning (少样本学习) :

提供少量示例，引导模型理解任务格式

few_shot_prompt = ""

任务：将自然语言查询转换为SQL语句

示例1：

问题：列出所有年龄大于25岁的用户

SQL: SELECT * FROM users WHERE age > 25;

示例2：

问题：查找名字包含"Alice"的用户邮箱

SQL: SELECT email FROM users WHERE name LIKE '%Alice%';

示例3：

问题：统计每个部门的员工数量

SQL: SELECT department_id, COUNT(*) as employee_count FROM employees
GROUP BY department_id;

现在请转换以下问题：

问题：找出购买金额超过1000元的客户姓名和电话

SQL:

""

Few-shot的优势：

☐ 任务定义更清晰

☐ 输出格式更一致

☐ 减少幻觉 (Hallucination)

Chain-of-Thought (思维链):

```
# 引导模型逐步推理，而非直接给答案
cot_prompts = {
    "数学推理": """
问题：小明有15个苹果，给了小红3个，又买了7个，然后吃掉了4个。
请问小明现在有多少个苹果？

让我们一步步思考：
步骤1：初始苹果数量 = 15个
步骤2：给小红后剩余 =  $15 - 3 = 12$ 个
步骤3：购买后总数 =  $12 + 7 = 19$ 个
步骤4：吃掉后剩余 =  $19 - 4 = 15$ 个

答案：小明现在有15个苹果
""",
    "逻辑推理": """
问题：如果所有的A都是B，所有的B都是C，那么所有的A都是C吗？

让我们一步步思考：
事实1：所有的A都是B ( $A \rightarrow B$ )
事实2：所有的B都是C ( $B \rightarrow C$ )
推导：既然A中的元素都在B中，而B中的元素都在C中，
        那么A中的元素必然也在C中。

结论：是的，所有的A都是C。这是传递关系的体现。
""",
    *
}
```

□ 2.2 高级Prompt技巧

角色设定 (Persona):

```
persona_prompts = {
```

```
    "专业顾问": """
```

```
    你现在是一位拥有20年经验的资深Python技术架构师。
```

```
    你精通系统设计、性能优化、代码审查和团队管理。
```

```
    你的回答应该:
```

- 专业且准确
- 提供最佳实践建议
- 考虑生产环境的各种情况
- 给出具体的代码示例

```
    问题: 如何设计一个高并发的秒杀系统?
```

```
    """,
```

```
    "创意助手": """
```

```
    你是一位富有想象力的创意写作大师, 擅长用生动的语言讲述故事。
```

```
    你的风格特点:
```

- 用词优美, 善用比喻和意象
- 能引发读者共鸣
- 故事结构精巧, 有起承转合

```
    请以"一颗种子的旅行"为题, 写一段300字的创意短文:
```

```
    """,
```

```
    "严格导师": """
```

```
    你是一位严格的代码审查员, 对代码质量有极高要求。
```

```
    你会指出每一个潜在的问题, 包括但不限于:
```

- 性能瓶颈
- 安全漏洞
- 可维护性问题
- 错误处理不足

```
    请审查以下代码并给出改进意见:
```

```
    ```python
```

```
 def get_user(user_id):
```

```
 user = db.execute(f"SELECT * FROM users WHERE id={user_id}")
```

```
 return user[0]
```

```
 "" }
```

**\*\*输出格式控制\*\*:**

```
```python
format_control = {
    "JSON格式": """
请以JSON格式返回以下信息：
{
    "name": "产品名称",
    "price": 价格数字,
    "features": ["特性1", "特性2"],
    "rating": 1-5的评分
}
*

```

产品信息：MacBook Pro 16英寸 M3芯片 36GB内存 512GB存储
"""

“结构化报告”： “”
请按以下模板生成数据分析报告：

【报告标题】

一、数据概览

- 数据量：
- 时间范围：
- 主要字段：

二、关键发现

发现1

- 数据支持：
- 业务含义：

发现2

...

三、结论与建议

- 1.
- 2.

四、风险提示

-
"""

“Markdown表格”： “”
将以下产品信息转换为Markdown表格：

产品列表：

1. iPhone 15 Pro - *8999 - A17 Pro芯片-钛金属设计
2. MacBook Air M3 - *8999-M3芯片-15英寸屏幕

3. AirPods Pro 2 - *1899-自适应音频-USB-C充电

4. Apple Watch Ultra 2 - *6499-双频GPS-钛金属表壳

请生成包含：序号、产品名称、价格（*）、主要特性的表格。

约束条件设定：

```
constraint_prompts = {
    "长度限制": ""
    请用不超过50个字介绍量子计算的基本原理：
    "",

    "目标受众调整": ""
    请分别针对以下三类人群解释"机器学习"：

    1. 小学生（10岁）：
    2. 高中生（16岁）：
    3. 计算机专业大学生：
    "",

    "禁止事项": ""
    请推荐5本关于人工智能的经典书籍。
    要求：
    - 不要推荐《人工智能：现代方法》（太经典了）
    - 至少包含1本中文著作
    - 每本书附上推荐理由（不超过30字）
    - 按推荐程度降序排列
    "",

    "思维框架": ""
    请使用SWOT分析法评估"在传统企业中引入AI Agent"这一决策：

    Strengths（优势）：
    Weaknesses（劣势）：
    Opportunities（机会）：
    Threats（威胁）：
    最终建议：
    ""
    *
```

3. DeepSeek API 实战（2小时）

3.1 API基础配置

```
# config.py
import os
from dataclasses import dataclass
from typing import Optional

@dataclass
class LLMConfig:
    """大模型配置类"""

    # DeepSeek配置
    deepseek_api_key: str = os.getenv("DEEPSEEK_API_KEY", "")
    deepseek_base_url: str = "https://api.deepseek.com/v1"

    # 豆包(火山引擎)配置
    doubao_api_key: str = os.getenv("DOUBAO_API_KEY", "")
    doubao_base_url: str = "https://ark.cn-beijing.volces.com/api/v3"

    # 通用配置
    default_model: str = "deepseek-chat"
    max_tokens: int = 4096
    temperature: float = 0.7
    timeout: int = 60

# .env文件
DEEPSEEK_API_KEY=sk-your-deepseek-api-key-here
DOUBAO_API_KEY=your-doubao-api-key-here
DEFAULT_MODEL=deepseek-chat
```

□ 3.2 DeepSeek API封装

```

# llm_clients/deepseek_client.py
import httpx
import json
from typing import AsyncIterator, Dict, List, Any, Optional
from config import LLMConfig

class DeepSeekClient:
    """DeepSeek API客户端"""

    def __init__(self, config: LLMConfig):
        self.config = config
        self.client = httpx.AsyncClient(
            base_url=config.deepseek_base_url,
            headers={
                "Authorization": f"Bearer {config.deepseek_api_key}",
                "Content-Type": "application/json"
            },
            *,
            timeout=config.timeout
        )

    async def chat(
        self,
        messages: List[Dict[str, str]],
        model: str = None,
        temperature: float = None,
        max_tokens: int = None,
        stream: bool = False,
        **kwargs
    ) -> Dict[str, Any]:
        """
        对话接口

        Args:
            messages: 对话消息列表
            model: 模型名称（默认deepseek-chat）
            temperature: 温度参数（0-2，越高越随机）
            max_tokens: 最大生成token数
            stream: 是否流式输出

        Returns:
            API响应字典
        """
        payload = {
            "model": model or self.config.default_model,
            "messages": messages,
            "temperature": temperature or self.config.temperature,
            "max_tokens": max_tokens or self.config.max_tokens,
            "stream": stream,
            **kwargs
        }

```



```
response = await self.client.post("/chat/completions", json=payload)
```

```
response.raise_for_status()
```



```
return response.json()
```



```
async def chat_stream(
```


self,

```
messages: List[Dict[str, str]],
```

****kwargs**

```
) -> AsyncIterator[str]:
```


流式对话接口（逐token返回）


```
kwargs["stream"] = True
```



```
response = await self.client.post(
```

"/chat/completions",

json={

```
"model": kwargs.pop("model", self.config.default_model),
```

"messages": messages,

```
"temperature": kwargs.pop("temperature", self.config.temperature),
```

```
"max_tokens": kwargs.pop("max_tokens", self.config.max_tokens),
```

"stream": True,

****kwargs**

timeout=None # 流式请求不设置超时


```
buffer = ""
```

```
async for line in response.aiter_lines():
```

```
if line.startswith("data: ") and line != "data: [DONE]":
```



```
data = json.loads(line[6:])
```

```
if data.get("choices"):
```

```
delta = data["choices"][0].get("delta", {*)
```

```
content = delta.get("content", "")
```

```
if content:
```

```
buffer += content
```

yield content


```
async def simple_chat(self, user_message: str, system_prompt: str = None) -> str:
```


简化的单轮对话接口

Args:

user_message: 用户消息

system_prompt: 系统提示词（可选）

Returns:

模型回复文本


```
messages = []
```



```
if system_prompt:
```

```
messages.append({"role": "system", "content": system_prompt*})
```



```
messages.append({"role": "user", "content": user_message*})
```



```
result = await self.chat(messages)
```



```
return result["choices"][0]["message"]["content"]
```



```
async def analyze_sentiment(self, text: str) -> Dict[str, Any]:
```


情感分析（封装好的专用功能）


```
prompt = f"""请分析以下文本的情感倾向。
```


要求：

1. 判断情感类别（正面/负面/中性）

2. 给出置信度（0-1之间的数值）

3. 提取关键情感词汇

4. 简要说明理由

请以JSON格式返回：

```
{{"sentiment": "类别", "confidence": 0.95, "keywords": ["词1", "词2"],
```

"reason": "原因"**

待分析文本：

```
{text*"""
```



```
response = await self.simple_chat(prompt)
```


try:

提取JSON部分

```
import re
```

```
json_match = re.search(r'\{.*\}', response, re.DOTALL)
```

```
if json_match:
```



```
return json.loads(json_match.group())
```

except:

pass


```
return {"raw_response": response*
```



```
async def summarize_text(
```

self,

text: str,

```
max_length: int = 200,
```

```
style: str = "concise"
```

) -> str:

1111

文本摘要

Args:

text: 原文

max_length: 摘要最大长度

style: 摘要风格 (concise/detailed/bullet-points)


```
style_instructions = {
```

"concise": "简洁明了，突出核心观点",

"detailed": "保留主要细节和论据",

"bullet-points": "使用项目符号列表形式"


```
prompt = f"""请对以下文本进行摘要。
```


要求：

- 长度控制在 {max_length*字以内

- 风格: `{style_instructions.get(style, style_instructions['concise'])}`

- 保持原文的关键信息和逻辑结构

- 不添加原文没有的信息

原文：

```
{text[:8000]}* # 限制输入长度避免超出context
```


摘要: ""


```
return await self.simple_chat(prompt)
```



```
async def translate_text(
```

self,

text: str,

```
source_lang: str = "auto",
```

```
target_lang: str = "English"
```

) -> str:

文本翻译


```
prompt = f"""请将以下文本从{source_lang*}翻译为{target_lang*}。
```


要求：

- 准确传达原意

- 符合目标语言的语法和表达习惯

- 保持专业术语的一致性

- 如果有文化差异，适当进行本地化调整

原文：

{text*

译文: ""


```
return await self.simple_chat(prompt)
```



```
async def extract_structured_data(
```

self,

text: str,

schema: Dict[str, Any]

```
) -> Dict[str, Any]:
```


结构化信息提取

Args:

text: 原始文本

schema: 期望输出的JSON Schema

1111

```
prompt = f"""请从以下文本中提取结构化信息。
```


输出格式要求（JSON Schema）：

```
{json.dumps(schema, ensure_ascii=False, indent=2)*
```


原始文本：

{text*

请严格按照上述格式提取信息，无法确定的信息填null：

提取结果: ""


```
response = await self.simple_chat(prompt)
```


try:

```
import re
```

```
json_match = re.search(r'\{.*\}', response, re.DOTALL)
```

```
if json_match:
```



```
return json.loads(json_match.group())
```

except:

pass


```
return {"raw_response": response, "extraction_failed": True*
```



```
async def close(self):
```

"""关闭客户端连接"""


```
await self.client.aclose()
```


使用示例

```
async def main():
```

```
config = LLMConfig()
```

```
client = DeepSeekClient(config)
```


try:

简单对话

```
reply = await client.simple_chat(
```

"用Python写一个计算斐波那契数列第n项的函数"


```
print("=== 代码生成 ===")
```

```
print(reply)
```


情感分析

```
sentiment = await client.analyze_sentiment(
```

"这个产品的用户体验太差了，界面复杂难用，而且经常崩溃！"


```
print("\n=== 情感分析 ===")
```

```
print(sentiment)
```


文本摘要


```
summary = await client.summarize_text("""
```

人工智能（AI）正在深刻改变着我们的生活方式和工作模式。从智能手机中的语音助手，

到医疗诊断系统，再到自动驾驶汽车，AI的应用无处不在。机器学习作为AI的核心技术，

通过从大量数据中学习模式和规律，使得计算机能够执行需要智能的任务。

深度学习的突破进一步推动了AI的发展，特别是在图像识别和自然语言处理领域。

然而，AI的发展也带来了挑战，包括就业影响、隐私问题和算法偏见等。

未来，我们需要在推动技术创新的同时，确保AI的发展符合伦理规范，造福人类社会。

""")


```
print("\n=== 文本摘要 ===")
```

```
print(summary)
```


流式输出

```
print("\n=== 流式输出 ===")
```

```
async for chunk in client.chat_stream([
```

```
{"role": "user", "content": "讲一个关于程序员的小故事"}
```



```
print(chunk, end="", flush=True)
```

```
print() # 换行
```


finally:

```
await client.close()
```



```
if __name__ == "__main__":
```

```
import asyncio
```



```
asyncio.run(main())
```

□4. 豆包(火山引擎) API 集成 (1.5小时)

□4.1 豆包API特性

豆包大模型优势:

✓ 多模态支持 - 文本+图片理解 ✓ 中文优化 - 在中文场景表现优秀 ✓ 角色扮演 - 强大的指令跟随能力 ✓ 知识时效性 - 接入实时搜索(部分模型) ✓ 性价比高 - 相比国外模型价格更低

可用模型:

```
DOUBAO_MODELS = {  
    # 对话模型  
    "doubao-pro-32k": {  
        "description": "豆包Pro 32K版本",  
        "context_length": 32768,  
        "strengths": ["复杂推理", "长文本理解", "代码生成"]  
    },  
    "doubao-lite-32k": {  
        "description": "豆包Lite轻量版",  
        "context_length": 32768,  
        "strengths": ["快速响应", "成本敏感场景", "简单任务"]  
    },  
    # 专用模型  
    "doubao-vision": {  
        "description": "视觉理解模型",  
        "capabilities": ["图像描述", "OCR", "图表理解", "视觉问答"]  
    },  
    # 嵌入模型(用于RAG)  
    "doubao-embedding": {  
        "description": "文本嵌入模型",  
        "dimension": 1024,  
        "use_case": ["语义搜索", "文本聚类", "RAG检索"]  
    }  
}
```

□4.2 豆包客户端实现

```

# llm_clients/doubao_client.py
import httpx
import base64
import json
from typing import List, Dict, Any, Optional, AsyncIterator
from config import LLMConfig

class DoubaoClient:
    """豆包（火山引擎）API客户端"""

    def __init__(self, config: LLMConfig):
        self.config = config
        self.client = httpx.AsyncClient(
            base_url=config.doubao_base_url,
            headers={
                "Authorization": f"Bearer {config.doubao_api_key}",
                "Content-Type": "application/json"
            },
            *,
            timeout=config.timeout
        )

    async def chat(
        self,
        messages: List[Dict[str, Any]],
        model: str = "doubao-pro-32k",
        **kwargs
    ) -> Dict[str, Any]:
        """
        豆包对话接口

        注意：豆包API兼容OpenAI格式，但有一些特殊参数
        """
        payload = {
            "model": model,
            "messages": messages,
            "temperature": kwargs.get("temperature", 0.7),
            "max_tokens": kwargs.get("max_tokens", 4096),
            "top_p": kwargs.get("top_p", 0.9),  # 豆包特有参数
            **{k: v for k, v in kwargs.items()
               if k not in ['temperature', 'max_tokens', 'top_p']}
        }

        response = await self.client.post("/chat/completions", json=payload)
        response.raise_for_status()
        return response.json()

    async def vision_chat(
        self,
        image_path: str,

```

question: str,

```
model: str = "doubao-vision"
```

) -> str:

1111

视觉理解（图文对话）

Args:

image_path: 图片路径或URL

question: 关于图片的问题

读取并编码图片

```
if image_path.startswith(('http://', 'https://')):
```

```
image_url = image_path
```



```
media_type = "image/jpeg"
```

else:

```
with open(image_path, 'rb') as f:
```

```
image_data = base64.standard_b64encode(f.read()).decode('utf-8')
```

```
image_url = f"data:image/jpeg;base64,{image_data}"
```

```
media_type = "image/jpeg"
```



```
messages = [
```


"role": "user",

"content": [

"type": "image_url",

"image_url": {"url": image_url*

"type": "text",

"text": question


```
result = await self.chat(messages, model=model)
```



```
return result["choices"][0]["message"]["content"]
```



```
async def role_play(
```

self,

character: str,

scenario: str,

```
user_input: str,
```

```
history: List[Dict] = None
```


) -> str:

角色扮演对话

Args:

character: 角色设定 (如"经验丰富的Python导师")

scenario: 场景描述

user_input: 用户输入

history: 历史对话记录


```
system_prompt = f"""你现在是{character*。
```


场景设定：

{scenario*

你的特点：

1. 完全沉浸在角色中，不脱离人设

2. 回答风格符合角色特征

3. 能够进行自然的互动

4. 必要时可以主动提问引导对话

请根据用户的输入，以角色的身份回应。"""


```
messages = [{"role": "system", "content": system_prompt*}]
```



```
if history:
```

```
messages.extend(history[-10:]) # 只保留最近10轮
```



```
messages.append({"role": "user", "content": user_input*})
```



```
result = await self.chat(messages)
```

```
return result["choices"][0]["message"]["content"]
```



```
async def knowledge_qa(
```

self,

question: str,

```
domain: str = "general"
```

```
) -> Dict[str, Any]:
```

1111

知识问答（利用豆包的知识库）


```
prompts = {
```

"tech": f""你是一位科技领域的专家，对最新的技术趋势、编程语言、

开源项目等有深入了解。请准确回答以下技术问题。如果不确定，请明确说明。

问题: {question*""},

"business": f""你是一位商业分析师，擅长市场分析、商业模式解读、

行业趋势预测等。请从专业角度回答以下商业相关问题。

问题: {question*""},

"general": f""请基于你的知识库，全面准确地回答以下问题。

如果涉及时效性强的信息，请注意说明信息的时效性。

问题: {question*}""


```
prompt = prompts.get(domain, prompts["general"])
```



```
result = await self.simple_chat(prompt)
```



```
return {
```

"answer": result,

"domain": domain,

"model_used": "doubao-pro-32k"


```
async def creative_writing(
```

self,

topic: str,

```
style: str = "professional",
```



```
length: str = "medium",
```

****constraints**

) -> str:

创意写作


```
style_guide = {
```

"professional": "正式、严谨、逻辑清晰",

"casual": "轻松、幽默、口语化",

"academic": "学术性强、引用充分、论证严密",

"storytelling": "引人入胜、富有想象力、情节生动"


```
length_guide = {
```

"short": "200-400字",

"medium": "600-1000字",

"long": "1500-2500字"


```
constraint_str = "\n".join([f"- {k*}: {v*}" for k, v in constraints.items()])
```



```
prompt = f"""请以"{style_guide.get(style, style_guide['profe
```

ssional'])*"的风格,

写一篇关于"{topic*}"的文章。

要求：

- 字数范围: {length_guide.get(length, length_guide['medium'])}*

- 风格: {style*}

```
{f'- 特殊要求: \n{constraint_str*' if constraints else ''*
```


文章内容: ""


```
return await self.simple_chat(prompt)
```



```
async def simple_chat(self, message: str, system_prompt: str = None) -> str:
```

""简化的单轮对话""

```
messages = []
```

```
if system_prompt:
```

```
messages.append({"role": "system", "content": system_prompt*})
```

```
messages.append({"role": "user", "content": message*})
```



```
result = await self.chat(messages)
```

```
return result["choices"][0]["message"]["content"]
```



```
async def close(self):
```

```
await self.client.aclose()
```


使用示例

```
async def doubao_demo():
```

```
config = LLMConfig()
```

```
client = DubaoClient(config)
```


try:

1. 普通对话

```
print("=== 豆包对话 ===")
```



```
reply = await client.simple_chat("介绍一下中国的四大发明")
```

```
print(reply[:200], "...")
```


2. 角色扮演

```
print("\n=== 角色扮演：面试官 ===")
```

```
interviewer_reply = await client.role_play(
```

character="严格的互联网公司技术面试官",

scenario="你正在面试一位申请高级Python工程师职位的候选人",


```
user_input="请你先做一个自我介绍"
```



```
print(interviewer_reply)
```


3. 创意写作

```
print("\n=== 创意写作 ===")
```

```
article = await client.creative_writing(
```

topic="未来城市中的AI生活",

style="storytelling",

length="short"


```
print(article)
```


4. 视觉理解（如果有图片）

```
# print("\n== 视觉理解 ==")
```

```
# vision_result = await client.vision_chat(
```



```
# image_path="./test_image.jpg",
```

question="这张图片里有什么？"

)

```
# print(vision_result)
```


finally:

```
await client.close()
```



```
if __name__ == "__main__":
```

```
import asyncio
```

```
asyncio.run(doubao_demo())
```

□5. 统一LLM服务层（1小时）

□5.1 抽象基类与工厂模式

```

# llm_service/base.py
from abc import ABC, abstractmethod
from typing import List, Dict, Any, AsyncIterator, Optional
from dataclasses import dataclass
from enum import Enum

class LLMPProvider(Enum):
    """LLM提供商枚举"""
    DEEPSEEK = "deepseek"
    DOUBAO = "doubao"
    OPENAI = "openai" # 预留

@dataclass
class ChatMessage:
    """消息数据类"""
    role: str # system/user/assistant
    content: str

@dataclass
class ChatCompletion:
    """聊天完成结果"""
    content: str
    model: str
    provider: LLMPProvider
    usage: Optional[Dict[str, int]] = None
    finish_reason: Optional[str] = None

class BaseLLMClient(ABC):
    """LLM客户端抽象基类"""

    @abstractmethod
    async def chat(
        self,
        messages: List[ChatMessage],
        **kwargs
    ) -> ChatCompletion:
        """发送聊天请求"""
        pass

    @abstractmethod
    async def chat_stream(
        self,
        messages: List[ChatMessage],
        **kwargs
    ) -> AsyncIterator[str]:
        """流式聊天"""
        pass

    @abstractmethod

```

```
async def simple_chat()
```

self,

message: str,


```
system_prompt: str = None
```

) -> str:

""简单对话""

pass

@abstractmethod

```
async def close(self):
```

""关闭连接""

pass

@property

@abstractmethod

```
def provider(self) -> LLMPProvider:
```

"""返回提供商类型"""

pass

□ 5.2 统一服务实现


```

# llm_service/unified_service.py
from typing import List, Dict, Any, Optional, AsyncIterator
from .base import BaseLLMClient, ChatMessage, ChatCompletion, LLMPProvider
from ..llm_clients.deepseek_client import DeepSeekClient
from ..llm_clients.doubao_client import DoubaoClient
from ..config import LLMConfig

class UnifiedLLMService:
    """统一的大语言模型服务"""

    def __init__(self, config: LLMConfig):
        self.config = config
        self._clients: Dict[LLMProvider, BaseLLMClient] = {}
        self._initialize_clients()

    def _initialize_clients(self):
        """初始化所有可用的客户端"""
        if self.config.deepseek_api_key:
            self._clients[LLMProvider.DEEPSEEK] = DeepSeekClient(self.config)

        if self.config.doubao_api_key:
            self._clients[LLMProvider.DOUBAO] = DoubaoClient(self.config)

    def get_client(self, provider: LLMProvider = None) -> BaseLLMClient:
        """获取指定提供商的客户端"""
        if provider is None:
            provider = self._get_default_provider()

        client = self._clients.get(provider)
        if not client:
            raise ValueError(f"未配置 {provider.value*} 的API Key")

        return client

    def _get_default_provider(self) -> LLMProvider:
        """获取默认提供商"""
        preferred_order = [
            LLMProvider.DEEPSEEK,
            LLMProvider.DOUBAO
        ]

        for provider in preferred_order:
            if provider in self._clients:
                return provider

        raise RuntimeError("未配置任何LLM提供商")

    async def chat(
        self,

```

```
messages: List[ChatMessage],
```

```
provider: LLMProvider = None,
```

****kwargs**

) -> ChatCompletion:

""统一聊天接口""

```
client = self.get_client(provider)
```

```
return await client.chat(messages, **kwargs)
```



```
async def smart_route(
```

self,

```
messages: List[ChatMessage],
```

```
task_type: str = "general"
```

) -> ChatCompletion:

智能路由：根据任务类型自动选择最优模型

Args:

messages: 消息列表

task_type: 任务类型

- general: 通用对话 (DeepSeek)

- coding: 代码相关 (DeepSeek)

- creative: 创意写作 (豆包)

- vision: 视觉理解 (豆包)

- reasoning: 复杂推理 (DeepSeek)

- chinese_optimized: 中文优化 (豆包)

1111

```
routing_table = {
```

```
"general": LLMProvider.DEEPSEEK,
```

```
"coding": LLMProvider.DEEPSEEK,
```

"creative": LLMProvider.DOUBAO,

"vision": LLMProvider.DOUBAO,

"reasoning": LLMPProvider.DEEPSEEK,

"chinese_optimized": LLMProvider.DOUBAO,

"role_play": LLMPProvider.DOUBAO,

"analysis": LLMProvider.DEEPSEEK


```
selected_provider = routing_table.get(task_type, LLMPProvider.DEEPSEEK)
```

```
client = self.get_client(selected_provider)
```



```
return await client.chat(messages)
```



```
async def fallback_chat(
```

self,

```
messages: List[ChatMessage],
```

****kwargs**

) -> ChatCompletion:

带故障转移的聊天（主服务商失败时自动切换）


```
providers = list(self._clients.keys())
```



```
last_error = None
```

for provider in providers:

try:


```
client = self.get_client(provider)
```

```
return await client.chat(messages, **kwargs)
```

```
except Exception as e:
```

```
last_error = e
```

```
print(f"□{provider.value* 调用失败: {e*}, 尝试下一个...")
```

continue


```
raise RuntimeError(f"所有LLM提供商均不可用。最后一个错误: {last_error*}")
```



```
async def batch_process(
```

self,

```
tasks: List[List[ChatMessage]],
```

```
provider: LLMProvider = None,
```

```
max_concurrent: int = 5
```

```
) -> List[ChatCompletion]:
```

1111

批量处理（并发控制）


```
import asyncio
```

```
from asyncio import Semaphore
```



```
semaphore = Semaphore(max_concurrent)
```

```
results = [None] * len(tasks)
```



```
async def process_one(index: int, task: List[ChatMessage]):
```

async with semaphore:

```
result = await self.chat(task, provider=provider)
```

```
results[index] = result
```


并发执行所有任务

```
await asyncio.gather(*[
```

```
process_one(i, task) for i, task in enumerate(tasks)
```


return results


```
async def close_all(self):
```

""关闭所有客户端连接""

```
for client in self._clients.values():
```

```
await client.close()
```


FastAPI 依赖注入

```
def get_llm_service():
```

"""获取LLM服务实例（FastAPI Depends使用）"""

```
config = LLMConfig()
```

```
service = UnifiedLLMService(config)
```

try:

yield service

finally:

```
asyncio.get_event_loop().run_until_complete(service.close_all())
```

□ 5.3 FastAPI集成示例

```

# routers/ai_router.py
from fastapi import APIRouter, HTTPException, BackgroundTasks
from pydantic import BaseModel, Field
from typing import List, Optional
from llm_service.unified_service import UnifiedLLMService, ChatMessage
import uuid
import time

router = APIRouter(prefix="/api/ai", tags=["AI Features"])

# Pydantic模型
class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=10000)
    system_prompt: Optional[str] = None
    provider: Optional[str] = None # "deepseek" or "doubao"
    stream: bool = False
    temperature: Optional[float] = Field(0.7, ge=0, le=2)
    max_tokens: Optional[int] = Field(2048, ge=1, max_value=8192)

class BatchRequest(BaseModel):
    tasks: List[str] = Field(..., min_items=1, max_items=20)
    system_prompt: Optional[str] = None

class SummarizeRequest(BaseModel):
    text: str = Field(..., min_length=50, max_length=50000)
    style: str = Field("concise", pattern="^(concise|detailed|bullet-points)$")
    max_length: int = Field(200, ge=50, le=2000)

class SentimentRequest(BaseModel):
    texts: List[str] = Field(..., min_items=1, max_items=100)

# 路由端点
@router.post("/chat")
async def chat_endpoint(
    request: ChatRequest,
    service: UnifiedLLMService = Depends(get_llm_service)
):
    """通用AI对话接口"""
    request_id = str(uuid.uuid4())[:8]
    start_time = time.time()

    messages = []
    if request.system_prompt:
        messages.append(ChatMessage(role="system", content=request.system_prompt))
    messages.append(ChatMessage(role="user", content=request.message))

```


try:

```
if request.stream:
```


流式响应

```
from fastapi.responses import StreamingResponse
```



```
async def generate():
```

```
client = service.get_client()
```

```
full_content = ""
```

```
async for chunk in client.chat_stream(
```

```
[m.__dict__ for m in messages],
```



```
temperature=request.temperature,
```

```
max_tokens=request.max_tokens
```



```
full_content += chunk
```

```
yield f"data: {json.dumps({'chunk': chunk})*'\n\n'"
```



```
yield f"data: {json.dumps({'done': True, 'request_id': request_id})*\n
```

\n"


```
yield "data: [DONE]\n\n"
```



```
return StreamingResponse(
```

generate(),

media_type="text/event-stream"

else:

普通响应


```
result = await service.chat(
```

messages,

```
temperature=request.temperature,
```

```
max_tokens=request.max_tokens
```



```
processing_time = time.time() - start_time
```



```
return {
```

"request_id": request_id,

```
"reply": result.content,
```

```
"model": result.model,
```

```
"provider": result.provider.value,
```

```
"usage": result.usage,
```

```
"processing_time_ms": round(processing_time * 1000, 2)
```



```
except Exception as e:
```

```
raise HTTPException(status_code=500, detail=f"AI服务错误: {str
```

(e)*")


```
@router.post("/summarize")
```

```
async def summarize_endpoint(
```


request: SummarizeRequest,

```
service: UnifiedLLMService = Depends(get_llm_service)
```

):

"""文本摘要接口"""

```
client = service.get_client()
```



```
summary = await client.summarize_text(
```

```
text=request.text,
```



```
max_length=request.max_length,
```

style=request.style


```
return {
```

```
"original_length": len(request.text),
```

```
"summary_length": len(summary),
```

"summary": summary,


```
"compression_ratio": round(len(summary) / len(request.text), 2)
```



```
@router.post("/sentiment")
```

```
async def sentiment_analysis(
```

request: SentimentRequest,

background_tasks: BackgroundTasks,


```
service: UnifiedLLMService = Depends(get_llm_service)
```

):

"""批量情感分析接口"""

```
client = service.get_client()
```



```
results = []
```

```
for text in request.texts:
```

```
result = await client.analyze_sentiment(text)
```



```
results.append({
```

```
"text": text[:100] + "..." if len(text) > 100 else text,
```

"analysis": result

异步记录日志

```
def log_analysis():
```

可以保存到数据库用于后续分析

pass

```
background_tasks.add_task(log_analysis)
```



```
return {
```

```
"total_analyzed": len(results),
```

"results": results


```
@router.post("/translate")
```

```
async def translate_endpoint(
```

source_text: str,

```
target_language: str = "English",
```

```
source_language: str = "auto",
```

```
service: UnifiedLLMService = Depends(get_llm_service)
```

):

"""文本翻译接口"""

```
client = service.get_client()
```



```
translation = await client.translate_text(
```

text=source_text,

source_lang=source_language,

target_lang=target_language


```
return {
```

"source_text": source_text,

"target_language": target_language,

"translation": translation


```
@router.post("/generate-code")
```

```
async def code_generation(
```

description: str,

```
language: str = "python",
```

```
service: UnifiedLLMService = Depends(get_llm_service)
```

):

"""代码生成接口"""

```
system_prompt = f"""你是一位资深的[{language.upper()}]开发工程师。
```


请根据需求生成高质量、可运行的代码。

要求：

1. 代码要有完整的注释

2. 包含错误处理

3. 遵循 {language*} 的最佳实践

4. 如果适用，提供使用示例"""


```
client = service.get_client()
```



```
code = await client.simple_chat(
```

message=description,

system_prompt=system_prompt


```
return {
```

"language": language,

"description": description,

"generated_code": code


```
@router.get("/models")
```

```
async def list_models():
```

"""列出可用的AI模型"""


```
return {
```

```
"providers": {
```

"deepseek": {

```
"models": ["deepseek-chat", "deepseek-coder"],
```

```
"status": "available" if LLMConfig().deepseek_api_key else "not_config"
```

ured"

"doubao": {


```
"models": ["doubao-pro-32k", "doubao-lite-32k", "doubao-vision"],
```

```
"status": "available" if LLMConfig().doubao_api_key else "not_configur
```

ed"

"default_provider": "deepseek"


```
@router.get("/health")
```

```
async def ai_health_check():
```

""AI服务健康检查""

```
status = {
```

"service": "AI Module",

"status": "healthy",

```
"providers": {*
```



```
config = LLMConfig()
```



```
if config.deepseek_api_key:
```

```
status["providers"]["deepseek"] = "configured"
```

else:

```
status["providers"]["deepseek"] = "missing_api_key"
```



```
if config.doubao_api_key:
```

```
status["providers"]["doubao"] = "configured"
```

else:

```
status["providers"]["doubao"] = "missing_api_key"
```



```
if not config.deepseek_api_key and not config.doubao_api_key:
```

```
status["status"] = "degraded"
```



```
status["warning"] = "未配置任何AI提供商API密钥"
```


return status

*> 实践任务清单

□任务1：构建AI文本处理工具集（2小时）

功能需求：

创建一个Web界面，集成多种AI文本处理功能：

✓ 智能对话 - 支持多轮对话，可选择模型 ✓ 文本摘要 - 支持多种摘要风格 ✓ 情感分析 - 批量分析文本情感倾向 ✓ 代码生成 - 自然语言描述转代码 ✓ 多语言翻译 - 支持主流语言互译 ✓ 创意写作 - 多种文体和风格

前端页面示例（Vue3）：

```

<!-- AiTools.vue -->
<template>
  <div class="ai-tools">
    <h1>🧠 AI 文本处理工具箱</h1>

    <!-- 工具选择 -->
    <div class="tool-selector">
      <button
        v-for="tool in tools"
        :key="tool.id"
        :class="{ active: currentTool === tool.id}"
        @click="switchTool(tool.id)"
      >
        <span class="icon">{{ tool.icon }}</span>
        {{ tool.name }}
      </button>
    </div>

    <!-- 主工作区 -->
    <div class="workspace">
      <!-- 输入区 -->
      <div class="input-panel">
        <h3>{{ currentToolConfig.title }}</h3>
        <p class="description">{{ currentToolConfig.description }}</p>

        <textarea
          v-model="inputText"
          :placeholder="currentToolConfig.placeholder"
          rows="10"
        ></textarea>

        <!-- 参数配置 -->
        <div class="options" v-if="currentToolConfig.options">
          <label v-if="currentToolConfig.options.includes('style')">
            摘要风格:
            <select v-model="params.style">
              <option value="concise">简洁</option>
              <option value="detailed">详细</option>
              <option value="bullet-points">要点</option>
            </select>
          </label>

          <label v-if="currentToolConfig.options.includes('language')">
            目标语言:
            <select v-model="params.targetLanguage">
              <option value="English">英语</option>
              <option value="Japanese">日语</option>
              <option value="Korean">韩语</option>
              <option value="French">法语</option>
            </select>
          </label>
        </div>
      </div>
    </div>
  </div>

```

</select>

</label>


```
<label v-if="currentToolConfig.options.includes('codeLanguage')">
```

编程语言：

```
<select v-model="params.codeLanguage">
```

<option value="python">Python</option>

<option value="javascript">JavaScript</option>

<option value="java">Java</option>

<option value="go">Go</option>

</select>

</label>

</div>

<button

@click="processText"

:disabled="loading || !inputText"

class="process-btn"


```
{{ loading ? '* 处理中...' : `📄${currentToolConfig.action`
```

*、**

</button>

</div>

<!-- 输出区 -->

```
<div class="output-panel">
```

<h3>结果</h3>


```
<div class="result-content" v-if="outputText">
```

```
<pre>{{ outputText **</pre>
```

</div>

```
<div class="placeholder" v-else>
```

处理结果将显示在这里...

</div>


```
<div class="actions" v-if="outputText">
```


<button @click="copyResult">复制</button>

<button @click="clearAll">清空</button>

</div>

</div>

</div>

<!-- 对话历史（仅对话工具显示） -->

```
<div class="history" v-if="currentTool === 'chat' && chatHistory.length">
```


<h3>对话历史</h3>

```
<div class="messages">
```

<div

```
v-for="(msg, index) in chatHistory"
```

:key="index"

:class=["message", msg.role]"

{{ msg.role === 'user' ? '你' : 'AI' }}:

trong>

<p>{{ msg.content }}</p>

</div>

</div>

</div>

</div>

</template>

<script setup>

```
import { ref, computed * from 'vue'
```

```
import { aiApi * from '@api/ai'
```



```
const currentTool = ref('chat')
```

```
const inputText = ref('')
```

```
const outputText = ref('')
```

```
const loading = ref(false)
```



```
const chatHistory = ref([])
```



```
const params = ref({
```

style: 'concise',

targetLanguage: 'English',

codeLanguage: 'python'


```
const tools = [
```

```
{ id: 'chat', name: '🗨️智能对话', icon: '🗨️',
```

```
{ id: 'summarize', name: '📄 文本摘要', icon: '📄 *',
```

```
{ id: 'sentiment', name: '情感分析', icon: '📊',
```

```
{ id: 'translate', name: '📄 翻译', icon: '📄 *',
```

```
{ id: 'code', name: '□代码生成', icon: '□*',
```

```
{ id: 'creative', name: '□创意写作', icon: '□ *
```



```
const toolConfigs = {
```

chat: {

title: 'AI 对话助手',

description: '与AI进行多轮对话，获取答案和建议',

placeholder: '输入你的问题...',

action: '发送'

summarize: {

title: '文本摘要',

description: '自动提取文本核心内容，生成简洁摘要'，

placeholder: '粘贴需要摘要的长文本...',

```
options: ['style'],
```

action: '生成摘要'

sentiment: {

title: '情感分析',

description: '分析文本的情感倾向（正面/负面/中性）',

placeholder: '输入待分析的文本，每行一条... ',

action: '分析情感'

translate: {

title: '多语言翻译',

description: '高质量的AI翻译, 支持多种语言',

placeholder: '输入需要翻译的文本...',

```
options: ['language'],
```

action: '翻译'

code: {

title: '代码生成',

description: '用自然语言描述需求，AI帮你写代码'，

placeholder: '描述你想实现的功能，例如：用Python实现快速排序算法...',


```
options: ['codeLanguage'],
```

action: '生成代码'

creative: {

title: '创意写作',

description: '让AI帮你创作各类文案和故事',

placeholder: '描述写作主题和要求...',

action: '开始创作'


```
const currentToolConfig = computed(() => toolConfigs[currentTool.value])
```



```
function switchTool(toolId) {
```

```
currentTool.value = toolId
```

```
inputText.value = ''
```



```
outputText.value = ''
```



```
async function processText() {
```

```
loading.value = true
```



```
try {
```

```
switch (currentTool.value) {
```



```
case 'chat':
```

```
await handleChat()
```

break

```
case 'summarize':
```

```
await handleSummarize()
```

break

```
case 'sentiment':
```

```
await handleSentiment()
```


break

```
case 'translate':
```

```
await handleTranslate()
```

break

```
case 'code':
```

```
await handleCodeGeneration()
```

break

```
case 'creative':
```



```
await handleCreativeWriting()
```

break


```
* catch (error) {
```

```
outputText.value = `❏错误: ${error.message}*`
```

```
* finally {
```

```
loading.value = false
```



```
async function handleChat() {
```

```
chatHistory.value.push({ role: 'user', content: inputText.value *})
```



```
const response = await aiApi.chat(inputText.value)
```



```
chatHistory.value.push({ role: 'assistant', content: response.reply *})
```



```
outputText.value = response.reply
```

```
inputText.value = ''
```



```
async function handleSummarize() {
```

```
const response = await aiApi.summarize({
```

```
text: inputText.value,
```

style: params.value.style

*)

```
outputText.value = response.summary
```



```
// ... 其他处理函数类似
```



```
function copyResult() {
```

```
navigator.clipboard.writeText(outputText.value)
```



```
alert('已复制到剪贴板！')
```



```
function clearAll() {
```

```
inputText.value = ''
```

```
outputText.value = ''
```

```
chatHistory.value = []
```


</script>

<style scoped>

```
.ai-tools {
```

max-width: 1200px;

```
margin: 0 auto;
```

padding: 2rem;


```
.tool-selector {
```

```
display: flex;
```

gap: 1rem;

```
margin-bottom: 2rem;
```

```
flex-wrap: wrap;
```



```
.tool-selector button {
```

```
padding: 0.75rem 1.5rem;
```

border: 2px solid #ddd;

```
border-radius: 8px;
```

```
background: white;
```

```
cursor: pointer;
```

transition: all 0.3s;


```
.tool-selector button.active {
```

border-color: #007bff;

background: #e7f3ff;

color: #007bff;


```
.workspace {
```



```
display: grid;
```

```
grid-template-columns: 1fr 1fr;
```

gap: 2rem;


```
@media (max-width: 768px) {
```

```
.workspace {
```

```
grid-template-columns: 1fr;
```



```
.input-panel, .output-panel {
```

background: #f8f9fa;

padding: 1.5rem;

```
border-radius: 12px;
```


textarea {

width: 100%;

padding: 1rem;

border: 1px solid #ddd;

```
border-radius: 8px;
```

```
font-family: inherit;
```

```
resize: vertical;
```



```
.process-btn {
```

width: 100%;

```
padding: 1rem;
```

```
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
```

color: white;

border: none;


```
border-radius: 8px;
```

font-size: 1.1rem;

```
cursor: pointer;
```

```
margin-top: 1rem;
```



```
.process-btn:disabled {
```

opacity: 0.6;

cursor: not-allowed;


```
.result-content pre {
```

```
background: white;
```

```
padding: 1rem;
```

```
border-radius: 8px;
```

```
white-space: pre-wrap;
```


word-wrap: break-word;

</style>

检验标准：

- ☐ [] 所有6个AI工具都能正常调用
- ☐ [] 对话功能支持多轮交互
- ☐ [] 参数选择能正确传递给后端
- ☐ [] 加载状态和错误处理完善
- ☐ [] UI响应式适配移动端
- ☐ [] 结果可以一键复制

☐任务2：将AI能力集成到博客系统（1.5小时）

集成方案：

为Day2-4开发的博客系统添加AI增强功能：

- ☐ ✓ AI辅助写作 - 编辑文章时可调用AI续写、润色、纠错
- ☐ ✓ 智能标签推荐 - 发布文章时自动生成标签建议
- ☐ ✓ 评论情感分析 - 自动识别垃圾评论和负面情绪
- ☐ ✓ SEO优化建议 - AI分析文章并提供SEO改进建议
- ☐ ✓ 智能客服机器人 - 基于网站内容的FAQ问答机器人

关键代码片段：

```

# services/blog_ai_service.py
from typing import List, Dict
from llm_service.unified_service import UnifiedLLMService

class BlogAIService:
    """博客系统AI增强服务"""

    def __init__(self, llm_service: UnifiedLLMService):
        self.llm = llm_service

    async def suggest_tags(self, article_title: str, article_content: str) -> List[str]:
        """AI标签推荐"""
        prompt = f"""基于以下文章内容，推荐5-8个合适的标签。

标题: {article_title*}

内容摘要: {article_content[:2000]*}

要求:
1. 标签应该是热门搜索词
2. 包含2-4个字的技术关键词
3. 结合当前技术热点
4. 格式: 每个标签单独一行，不加任何符号

推荐的标签: """

        result = await self.llm.simple_chat(prompt)
        tags = [tag.strip().replace('#', ' ') for tag in result.split('\n') if tag.strip()]
        return tags[:8]

    async def analyze_comment(self, comment_text: str) -> Dict:
        """评论分析"""
        prompt = f"""分析以下博客评论:

评论内容: {comment_text*}

请判断:
1. 情感倾向 (positive/negative/neutral)
2. 是否为垃圾评论 (spam: true/false)
3. 是否需要人工审核 (need_review: true/false)
4. 回复建议 (可选, 针对有价值评论)

以JSON格式返回:
{"sentiment": "...", "spam": bool, "need_review": bool, "reply_suggestion": "..."}"""

        result = await self.llm.simple_chat(prompt)

```

解析JSON...

```
return {"raw_analysis": result*
```



```
async def seo_optimization(self, article: Dict) -> Dict:
```

""SEO优化建议""

prompt = f"""作为SEO专家，分析以下文章并提供优化建议：

标题: {article['title']}*

内容: {article['content'][:3000]}*

现有标签: {article.get('tags', [])}

请提供:

1. SEO标题建议（50-60字符，包含主要关键词）

2. Meta Description建议（150-160字符）

3. 关键词建议（5个主要关键词）

4. 内容优化建议（3-5条具体建议）

5. 内部链接建议（建议链接到哪些类型的文章）

以结构化格式返回："""


```
return await self.llm.simple_chat(prompt)
```



```
async def ai_continue_writing(self, context: str, style: str = "professional") ->
```

str:

""AI续写""

```
prompt = f"""你是专业的技术博客作者。请根据已有的开头，续写接下来的内容。
```


已有内容:

{context*

续写要求：

1. 保持相同的写作风格

2. 内容连贯自然

3. 技术准确性

4. 长度：300-500字

续写内容: ""


```
return await self.llm.simple_chat(prompt)
```



```
async def generate_faq_answer(self, question: str, site_context: str) -> str:
```

"""基于网站内容回答FAQ"""

```
prompt = f"""基于以下网站背景信息，回答用户问题。
```


网站相关信息:

```
{site_context[:2000]*
```


用户问题: {question*}

请给出准确、有帮助的回答。如果问题超出网站内容范围，请礼貌地说明。

回答: ""


```
return await self.llm.simple_chat(prompt)
```


FastAPI路由集成

```
@router.post("/articles/{article_id*}/ai-suggest-tags")
```

```
async def suggest_article_tags(article_id: int, db: Session = Depends(get_db)):
```



```
article = get_post(db, article_id)
```

if not article:

```
raise HTTPException(404, "文章不存在")
```



```
ai_service = BlogAIService(get_llm_service())
```

```
tags = await ai_service.suggest_tags(article.title, article.content)
```



```
return {"suggested_tags": tags*
```



```
@router.post("/comments/{comment_id*}/analyze")
```

```
async def analyze_comment(comment_id: int, db: Session = Depends(get_db)):
```

```
comment = get_comment(db, comment_id)
```

if not comment:

```
raise HTTPException(404, "评论不存在")
```



```
ai_service = BlogAIService(get_llm_service())
```

```
analysis = await ai_service.analyze_comment(comment.content)
```


保存分析结果

```
comment.sentiment = analysis.get('sentiment')
```

```
comment.is_spam = analysis.get('spam', False)
```

```
comment.needs_review = analysis.get('need_review', False)
```

```
db.commit()
```


return analysis

检验标准：

- ☐ [] 文章编辑页能调用AI续写和润色
- ☐ [] 发布文章时能获得标签推荐
- ☐ [] 评论能自动进行情感分析和垃圾检测
- ☐ [] SEO建议功能正常工作
- ☐ [] AI功能不影响原有系统的稳定性
- ☐ [] API调用次数和成本可控

** 推荐学习资源

☐ Prompt Engineering

- ☐ Learn Prompting - 最全面的Prompt教程
- ☐ OpenAI Prompt Engineering Guide - 官方指南
- ☐ Prompt Engineering Guide - 中英文双语

☐ DeepSeek资源

- ☐ DeepSeek API文档 - 完整API参考
- ☐ DeepSeek GitHub - 开源模型和代码
- ☐ DeepSeek论文 - 学术论文

☐ 豆包/火山引擎

- ☐ 火山引擎豆包API文档 - 官方文档
- ☐ 火山引擎控制台 - API Key管理
- ☐ 豆包模型介绍 - 模型对比

☐ LLM应用开发

- ☐ LangChain文档 - LLM应用框架
- ☐ LlamaIndex - 数据框架
- ☐ OpenAI Cookbook - 实用示例集合

✓ 今日自测题（精简版）

☐ 关键知识点检测

- ☐ Transformer相比RNN的主要优势？

- ☐ A. 参数更少
- ☐ B. 解决长距离依赖问题，支持并行计算
- ☐ C. 训练更快
- ☐ D. 占用内存更小

- ☐ Temperature参数的作用？
- ☐ A. 控制输出长度
- ☐ B. 控制随机性和创造性（越高越随机）
- ☐ C. 控制响应速度
- ☐ D. 控制成本

- ☐ Few-shot Learning的优势？
- ☐ A. 减少API调用次数
- ☐ B. 无需微调即可适应新任务
- ☐ C. 提高模型性能
- ☐ D. 降低成本

- ☐ Chain-of-Thought Prompting适用于？
- ☐ A. 翻译任务
- ☐ B. 数学推理、逻辑推理等复杂任务
- ☐ C. 图像生成
- ☐ D. 文本分类

- ☐ 为什么要在生产环境中实施故障转移（Fallback）？
- ☐ A. 降低成本
- ☐ B. 提高服务可用性和可靠性
- ☐ C. 加快响应速度
- ☐ D. 减少Token消耗

☐答案

- ☐ B - 注意力机制解决长距离依赖，矩阵运算可并行化GPU加速。
- ☐ B - Temperature控制概率分布的平滑程度，影响创造性。
- ☐ B - 通过示例让模型理解任务格式，无需重新训练。
- ☐ B - CoT通过分步推理提高复杂问题的准确率。
- ☐ B - 单一API可能宕机或限流，故障转移保证业务连续性。

** 今日总结

☐关键收获

- ☐ _____
- ☐ _____
- ☐ _____

☐ 明日预告（最后两天！）

Day 7-8: AI Agent核心技术与实战项目 将涵盖：

- ☐ Agent架构设计（ReAct、Plan-and-Execute）
- ☐ 工具调用机制（Function Calling）
- ☐ RAG（检索增强生成）实战
- ☐ 多Agent协作系统
- ☐ 毕业项目：智能文档助手AI Agent

准备工作：

- ☐ [] 复习前6天的所有代码
- ☐ [] 准备毕业项目的需求和设计思路
- ☐ [] 安装LangChain/LangGraph框架
- ☐ [] 准备测试用的文档数据

** 实战项目：统一LLM工具平台 ☐

☐ 项目概览

项目名称：AI Tools - DeepSeek/豆包统一调用平台 路径:06-AI基础理论与国内大模型应用/ai-tools/ 完成度：√ 100% 文件数：10个核心文件 代码量：1200+ 行 技术栈：Python + FastAPI + DeepSeek API + 豆包(Volcano) API

☐ 核心特性

√ DeepSeek客户端 - 对话/代码生成/情感分析/文本摘要 √ 豆包客户端 - 角色扮演/翻译/创意写作/视觉理解 √ 统一服务层 - 智能路由 + 自动故障转移 + 批量处理 √ Prompt模板库 - 摘要/翻译/代码生成场景模板 √ FastAPI集成 - RESTful API + 流式输出(SSE)

☐ 项目架构

```

ai-tools/
+-- deepseek_client.py      # DeepSeek API封装
|   +-- chat()              # 多轮对话
|   +-- generate_code()     # 代码生成
|   +-- analyze_sentiment() # 情感分析
|   +-- summarize_text()    # 文本摘要
|
+-- doubao_client.py        # 豆包(火山引擎)API封装
|   +-- role_play()         # 角色扮演对话
|   +-- translate()          # 多语言翻译
|   +-- creative_writing()   # 创意写作
|
+-- llm_service.py          # □统一LLM服务层
|   +-- _select_provider()   # 智能任务路由
|   +-- _fallback_chat()     # 自动故障转移
|   +-- batch_process()      # 并发批量处理
|
+-- prompts/                # Prompt模板库
|   +-- summarization.txt    # 摘要模板（简洁/详细/学术）
|   +-- translation.txt      # 翻译模板（通用/技术/商务）
|   +-- code_generation.txt  # 代码模板（基础/算法/API）
|
+-- test_ai_api.py          # 综合测试脚本

```

□核心功能演示

□1. 统一服务层智能路由

```

from llm_service import UnifiedLLMService

service = UnifiedLLMService(config)

# 自动选择最优模型
result = await service.smart_route(
    messages=[{"role": "user", "content": "写一个快排*"},
    task_type="coding" # * 自动路由到DeepSeek
)

result = await service.smart_route(
    messages=[{"role": "user", "content": "写一首诗*"},
    task_type="creative" # * 自动路由到豆包
)

```

□2. 故障转移机制

```
# 主服务商失败时自动切换
try:
    result = await service.fallback_chat(messages)
except Exception as e:
    # 自动尝试所有配置的Provider
    pass
```

❑ 3. FastAPI接口示例

```
POST /api/chat          # 通用AI对话
POST /api/summarize     # 文本摘要
POST /api/sentiment     # 情感分析
POST /api/translate     # 翻译
POST /api/generate-code # 代码生成
GET  /api/models        # 查看可用模型
GET  /api/health        # 服务健康检查
```

❑ 快速启动

```
# 1. 进入项目目录
cd 06-AI基础理论与国内大模型应用/ai-tools

# 2. 创建虚拟环境
python -m venv venv
venv\Scripts\activate # Windows
# source venv/bin/activate # Linux/macOS

# 3. 安装依赖
pip install -r requirements.txt

# 4. 配置API Key
set DEEPSEEK_API_KEY=sk-your-key-here # Windows
set DOUBAO_API_KEY=your-doubao-key-here
# 或创建 .env 文件

# 5. 运行测试
python test_ai_api.py
# 测试DeepSeek、豆包和统一服务的各项功能

# 6. 启动API服务
uvicorn main:app --reload --port 8000

# 7. 访问API文档
# http://localhost:8000/docs
```

□ Prompt模板示例

摘要模板 (prompts/summarization.txt):

[简洁模式]
请用2-3句话概括以下内容的核心观点：
{content*}

[详细模式]
请详细总结以下内容，包括：
1. 主要论点
2. 关键证据
3. 结论建议
{content*}

□ 验收标准

- ☐ [] DeepSeek对话功能正常
- ☐ [] 豆包角色扮演功能正常
- ☐ [] 统一服务智能路由正确
- ☐ [] 故障转移机制生效（模拟主服务商宕机）
- ☐ [] Prompt模板加载成功
- ☐ [] API响应时间 < 5秒（简单查询）
- ☐ [] 流式输出正常工作

*□ 模块导航

Day 5: 运维与部署实践 | Day 7-8: AI Agent核心技术与实战项目 | ✓ □ 返回课程首页

√* Day 6 完成！你已经掌握了AI应用开发的核心技能！

最后两天，我们将打造完整的AI Agent系统，完成从开发者到AI工程师的蜕变！